# Dynamic Waypoint Patrol + Blackboard

---

## What this project does

A patrol robot that navigates through a list of waypoints in a loop. The key feature is that the waypoints can be changed from outside while the robot is already running — no restart, no code change needed. This is done using two ROS2 concepts: **topics** and the **blackboard**.

---

## Core concepts

**Topic** — a communication pipe in ROS2. One node publishes to it, another subscribes. Fire and forget. The sender doesn't wait for a reply.

**Blackboard** — a shared memory space that all parts of the behavior tree can read from and write to. Think of it as a whiteboard in the middle of the room. Anyone can write on it, anyone can read it.

**Subscriber callback** — a function that automatically runs the moment a message arrives on a topic.

---

## The full flow

```
Your terminal
     ↓  ros2 topic pub
/patrol_waypoints topic
     ↓  message arrives
waypoint_callback()
     ↓  json.loads → _coords_to_poses
blackboard.waypoints updated
     ↓  next time send_next_goal runs
Nav2 gets new goal
     ↓
Robot moves
```

---

## What each part of the code does

`__init__` — sets up everything at startup: creates the navigator, creates the blackboard client, writes default waypoints to the blackboard, creates the subscriber, starts the 1-second timer.

`create_waypoints` — just the default coords used at startup. After this runs once, the code never touches hardcoded coords again.

`waypoint_callback` — fires when a message arrives on `/patrol_waypoints`. Parses the JSON string into a Python list, converts to PoseStamped objects, writes to blackboard. Robot picks these up after finishing its current waypoint.

`_coords_to_poses` — helper that converts raw `[x,y]` pairs into `PoseStamped` objects that Nav2 understands.

`control_loop` — runs every 1 second. Checks if current navigation is done. On success, moves index forward. On failure, retries up to 3 times then skips. On cancel, moves on. Calls `send_next_goal` after each result.

`send_next_goal` — reads current waypoints from blackboard, picks the one at `current_index`, stamps it with current time, fires it to Nav2 with the BT XML.

---

## The BT XML role

The XML file defines how Nav2 handles each individual navigation goal. It wraps the navigation in a `RecoveryNode` — if the path fails, it clears the costmap, spins 90°, waits 2 seconds, then retries. The Python code handles the *which waypoint* logic. The BT XML handles the *how to get there* logic.

---

## How to send new waypoints while running

bash

```
ros2 topic pub --once /patrol_waypoints std_msgs/String \
  'data: "[[1.0, 0.0], [2.0, 1.5], [0.5, 2.0]]"'
```

Format is always `[[x1,y1],[x2,y2],...]` as a JSON string.

---

## Clean version of the code (no logs, just the logic)

python

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from std_msgs.msg import String
from nav2_simple_commander.robot_navigator import BasicNavi
gator, TaskResult
import py_trees
import json


class PatrolBTNode(Node):
    def __init__(self):
        super().__init__('patrol_bt_node')

        self.navigator = BasicNavigator()
        self.navigator.waitUntilNav2Active()

        self.bt_xml = "/home/tasz/ros2_ws/src/patrol_blackb
oard/behavior_trees/patrol_blackboard.xml"

        self.blackboard = py_trees.blackboard.Client(name
="PatrolNode")
        self.blackboard.register_key("waypoints", access=py
_trees.common.Access.WRITE)
        self.blackboard.register_key("waypoints", access=py
_trees.common.Access.READ)
        self.blackboard.waypoints = self.create_waypoints()

        self.current_index = 0
        self.is_navigating = False
        self.retry_count = 0
        self.max_retries = 3

        self.create_subscription(String, '/patrol_waypoint
s', self.waypoint_callback, 10)
        self.create_timer(1.0, self.control_loop)

    def create_waypoints(self):
```

```
        return self._coords_to_poses([(0.5,0.0),(3.5,0.5),
(0.0,0.5),(0.0,0.0)])

    def waypoint_callback(self, msg):
        try:
            coords = json.loads(msg.data)
            self.blackboard.waypoints = self._coords_to_pos
es(coords)
            self.current_index = 0
            self.retry_count = 0
        except Exception:
            pass

    def _coords_to_poses(self, coords):
        poses = []
        for x, y in coords:
            p = PoseStamped()
            p.header.frame_id = "map"
            p.pose.position.x = float(x)
            p.pose.position.y = float(y)
            p.pose.orientation.w = 1.0
            poses.append(p)
        return poses

    def control_loop(self):
        if not self.is_navigating or not self.navigator.isT
askComplete():
            return

        result = self.navigator.getResult()
        waypoints = self.blackboard.waypoints

        if result == TaskResult.SUCCEEDED:
            self.current_index = (self.current_index + 1) %
len(waypoints)
            self.retry_count = 0
        elif result == TaskResult.FAILED:
            self.retry_count += 1
```

```
            if self.retry_count >= self.max_retries:
                self.current_index = (self.current_index +
1) % len(waypoints)
                self.retry_count = 0
        elif result == TaskResult.CANCELED:
            self.current_index = (self.current_index + 1) %
len(waypoints)
            self.retry_count = 0

        self.send_next_goal()

    def send_next_goal(self):
        goal = self.blackboard.waypoints[self.current_inde
x]
        goal.header.stamp = self.get_clock().now().to_msg()
        self.is_navigating = True
        self.navigator.goToPose(goal, behavior_tree=self.bt
_xml)


def main(args=None):
    rclpy.init(args=args)
    node = PatrolBTNode()
    node.send_next_goal()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()


if __name__ == "__main__":
    main()
```